CLIENTFLOW

Project Specification & Claude Code Practice Assignment

Project type: Client and Project Management SaaS

Difficulty: Beginner to Intermediate

Suggested duration: 2–3 weeks

Technology stack: Nuxt 3 · Laravel · Nuxt UI · MySQL

Important: Start from zero.
The learner must begin with an empty project directory and use Claude Code from the very first step: environment inspection, project initialization, architecture, implementation, testing, debugging, and documentation.

A practical assignment for learning AI-assisted software development.

1. Project Overview

ClientFlow is a web-based client and project management platform for freelancers, virtual assistants, and small VA agencies. It helps teams organize clients, manage projects, assign tasks, monitor deadlines, and track progress in one centralized application.

The application should have a clean, modern, responsive interface and follow sound software engineering practices.

2. Technology Stack

Layer	Technology	Purpose
Frontend	Nuxt 3, Vue 3 Composition API, TypeScript, Nuxt UI	Responsive interface and client-side application
Backend	Laravel REST API, Laravel Sanctum, Eloquent ORM	Business logic, authentication, validation, API services
Database	MySQL	Relational data storage
Development	Claude Code, Git, GitHub, ESLint, Prettier	AI-assisted development, version control, code quality

3. User Roles

Administrator

- Access the entire application.
- Manage users and team members.
- Create and manage clients, projects, and tasks.
- Assign team members and view all reports.

Project Manager

- Manage assigned clients and projects.
- Create and assign tasks.
- Monitor team progress and update project statuses.
- View project reports.

Team Member

- View assigned projects and tasks.
- Update task statuses.
- Add comments to tasks.
- Submit work updates.

4. Core Modules

4.1 Authentication

- User login and logout.
- Admin-controlled user registration.

- Forgot password and password reset.
- Role-based access control.
- Protected frontend routes and backend API endpoints.

4.2 Dashboard

- Show role-appropriate dashboard information.
- Total clients and active projects.
- Completed projects, pending tasks, overdue tasks, and tasks due today.
- Recent activities and project progress overview.

4.3 Client Management

- Create, view, edit, and archive clients.
- Search clients and filter by status.
- Store company name, contact person, email, phone, address, description, status, and creation date.

4.4 Project Management

- Create, edit, view, and archive projects.
- Assign project managers and team members.
- Set project start and end dates.
- Update project status and progress.
- Statuses: Planning, Active, On Hold, Completed, Cancelled.
- Store project name, client, description, dates, manager, members, status, and progress percentage.

4.5 Task Management

- Create, edit, and delete tasks according to permissions.
- Assign tasks to team members.
- Set priority, due date, and description.
- Update task status and add comments.
- Statuses: To Do, In Progress, Review, Completed.
- Priorities: Low, Medium, High, Urgent.
- Provide table and Kanban board views.

4.6 Task Comments and Activity Logs

- Allow team members to comment on tasks.
- Display comment author and timestamp.
- Record important activities, task status changes, and project updates.
- Display activity history.

4.7 Notifications

- Provide in-app notifications.
- Notify users about task assignments, approaching deadlines, comments, status changes, and project assignments.
- Allow users to mark notifications as read.

4.8 Reports

- Projects grouped by status.
- Tasks grouped by status.
- Completed versus pending tasks.
- Overdue tasks.
- Team member task completion summary.
- Date filters and basic charts.

5. Database Requirements

Create relational database tables. Suggested tables:

- users
- roles
- clients
- projects
- project_members
- tasks
- task_comments
- activity_logs
- notifications

Use appropriate foreign keys, indexes, timestamps, and database relationships. Avoid unnecessary data duplication. Document important relationship and design decisions.

6. UI/UX Requirements

- Use Nuxt UI components and maintain a modern SaaS dashboard appearance.
- Provide responsive sidebar navigation and a top navigation bar.
- Build reusable form components.
- Use modal dialogs for create and edit operations where appropriate.
- Provide tables with pagination, search, and filters.
- Include loading states, empty states, error handling, toast notifications, and confirmation dialogs before destructive actions.
- Maintain consistent spacing, typography, and component patterns.

7. Backend API Requirements

Create RESTful API endpoints following Laravel conventions. Example endpoints:

Resource	Method and endpoint
Authentication	POST /api/login POST /api/logout GET /api/user
Clients	GET /api/clients · POST /api/clients GET /api/clients/{id} · PUT /api/clients/{id} DELETE /api/clients/{id}

Resource	Method and endpoint
Projects	GET /api/projects · POST /api/projects GET /api/projects/{id} · PUT /api/projects/{id} DELETE /api/projects/{id}
Tasks	GET /api/tasks · POST /api/tasks GET /api/tasks/{id} · PUT /api/tasks/{id} DELETE /api/tasks/{id}

Implement validation, authorization, pagination, filtering, and consistent API responses.

8. Non-Functional Requirements

- Follow SOLID principles where appropriate.
- Use reusable frontend components.
- Use Laravel service classes when business logic warrants them.
- Keep controllers focused and maintainable.
- Follow consistent naming conventions and implement proper error handling.
- Prevent unauthorized data access on the backend.
- Use database transactions when appropriate.
- Avoid unnecessary API requests.
- Optimize database queries and prevent N+1 queries.

9. Deliverables

- Working Nuxt 3 frontend and Laravel REST API.
- MySQL database schema, migrations, and seeders.
- Authentication and role-based authorization.
- Responsive dashboard and complete CRUD operations.
- Task management, collaboration, notifications, and basic reporting.
- README with installation and setup instructions.
- GitHub repository with organized commits.

10. Definition of Done

The project is complete when users can log in, manage clients, create projects, assign tasks, collaborate through comments, receive notifications, and view reports. All protected operations must enforce authorization on the backend—not merely hide controls in the frontend.

11. Start-from-Scratch Development Workflow

Claude Code must be used from the first command. The learner should open Claude Code in an empty project directory and ask it to inspect the environment before creating files or installing dependencies.

Phase 1 – Environment inspection and project initialization

- Check installed versions of Node.js, npm or pnpm, PHP, Composer, and MySQL.
- Check whether required development tools are available; explain any missing prerequisites.
- Initialize the Nuxt 3 frontend and Laravel backend from scratch.
- Install and configure required dependencies, including Nuxt UI and Laravel Sanctum.
- Configure MySQL and environment variables without exposing secrets.
- Initialize Git and create a sensible .gitignore.

Phase 2 – Architecture and planning

- Inspect the newly created codebases and explain their structure.
- Propose frontend and backend folder organization.
- Design the database ERD and Eloquent relationships.
- Define API endpoints, validation rules, and authorization policies.
- Identify security, performance, and integration risks.
- Create a milestone-based implementation plan.

Phase 3 – Backend implementation

- Build migrations, models, factories, and seeders.
- Implement authentication, roles, policies, and protected API routes.
- Build client, project, team, and task APIs.
- Add comments, activity logs, notifications, and reporting endpoints.
- Add validation, pagination, filtering, and API resources.

Phase 4 – Frontend implementation

- Build the application layout, navigation, and dashboard.
- Implement authentication flows and protected routes.
- Build client, project, team, and task screens.
- Integrate API calls and handle loading, empty, and error states.
- Implement table and Kanban views, notifications, and reports.

Phase 5 – Testing, debugging, and review

- Generate and run Laravel feature tests and relevant frontend tests.
- Test role permissions and unauthorized access.
- Investigate bugs using logs, browser tools, and test output.
- Review database queries and fix N+1 issues.
- Ask Claude Code to review security, performance, and maintainability.

Phase 6 – Documentation and delivery

- Write setup and installation instructions.

- Document environment variables using an example file without real secrets.
- Document API behavior and key architectural decisions.
- Review Git history and ensure changes are organized.
- Prepare deployment notes and a production-readiness checklist.

12. Development Milestones

Milestone 1 – Environment and project setup

Inspect tools, create Nuxt and Laravel apps from scratch, configure MySQL, dependencies, environment files, and Git.

Milestone 2 – Database architecture

Design ERD, create migrations, Eloquent relationships, seeders, and factories.

Milestone 3 – Authentication and permissions

Implement Sanctum, login/logout, roles, policies, and protected API routes.

Milestone 4 – Core CRUD modules

Implement clients, projects, team members, and tasks.

Milestone 5 – Frontend integration

Build dashboard, tables, forms, modals, filters, pagination, and Kanban board.

Milestone 6 – Collaboration

Implement task comments, activity logs, and notifications.

Milestone 7 – Reports and testing

Build statistics and charts; test APIs, authorization, and frontend workflows.

Milestone 8 – Finalization

Fix bugs, refactor, optimize, document, and prepare delivery.

13. Claude Code Practical Challenges

Challenge	Exercise
1. Database design	Ask Claude to design the ERD. Review relationships before generating migrations.
2. Complete CRUD	Build Client Management from database through frontend.
3. Authorization	Ensure team members cannot access restricted projects or modify unauthorized records.
4. Debugging	Introduce a bug in a Laravel endpoint and use Claude Code to investigate it.
5. Refactoring	Improve a controller that contains too much business logic.
6. Performance	Identify and fix an N+1 query in project listing.
7. Testing	Create feature tests for CRUD and unauthorized access.
8. Code review	Ask Claude to review the implementation and prioritize issues.

14. Evaluation Criteria

Criteria	Weight
Understanding requirements	10%
Database design	15%
Laravel backend implementation	20%
Nuxt frontend implementation	20%
Claude Code usage	15%
Debugging and problem-solving	10%
Code quality and testing	10%
Total	**100%**

Evaluation principle: She should be able to explain the code Claude generates. When something breaks, she should understand how to investigate it rather than repeatedly asking Claude to fix everything.

15. First Claude Code Prompt

Run Claude Code from inside the empty project directory. Provide the full ClientFlow specification along with this prompt.

```
Act as a senior full-stack software architect and development partner.

We are going to build a SaaS application called ClientFlow from scratch.

Technology stack:
- Nuxt 3 with TypeScript for the frontend.
- Laravel REST API for the backend.
- Nuxt UI for the interface.
- MySQL for the database.
- Laravel Sanctum for authentication.

IMPORTANT RULES:
1. Help me initialize the entire project from an empty directory.
2. Do not assume Nuxt or Laravel is already installed or configured.
3. First inspect the environment and available tools.
4. Before executing commands, explain what they do.
5. Create both frontend and backend applications.
6. Configure MySQL and frontend/backend communication.
7. Follow maintainable architecture and security best practices.
8. Use reusable components, validation, authentication, and authorization.
9. Write tests alongside important features.
10. Maintain clear documentation.

WORKFLOW:
- Work in small, manageable milestones.
- Explain each milestone's objective and plan.
- Identify files to create or modify.
- Implement the functionality.
- Run relevant tests and review the result.
- Explain what was accomplished and what I should learn.
- Wait for my approval before moving to the next major milestone.

Do not generate the entire application in one go.

Start by inspecting my environment and checking the available development tools. Then propose
```

```
the initial project setup plan. I will provide the complete ClientFlow specification.

Help me build a production-quality application while teaching me good software engineering
practices and effective Claude Code usage.
```

16. Suggested AI-Powered Enhancements

After completing the core application, extend ClientFlow with one or more LLM-powered features:

- AI task description generator.
- AI project progress summarizer.
- AI-generated weekly reports.
- AI task prioritization assistant.

These extensions introduce API integrations, prompt engineering, and practical AI-powered application development.